

Java Interview Questions & Answers (2-3 Years Experience)

Source: foundit.in career-advice article — duplicate-topic questions merged/removed.

Q1. What is the difference between Method Overloading and Method Overriding in Java?

Method Overloading increases a program's readability. It has the same method name but different parameters, resolved within the same class.

Method Overriding provides a specific implementation of a method in a subclass. It has the same method name and the same parameters as the parent class method.

Q2. What do you mean by class loader in Java?

In the JVM, the class loader is a subsystem used for loading class files. When a program runs, it is first loaded by the class loader. There are three built-in class loaders:

- Bootstrap Classloader: the first class loader in the JVM; loads rt.jar, which contains the class files of Java Standard Edition (java.lang, java.net, java.util, java.io, java.sql, etc.).
- Extension Classloader: the child of Bootstrap; loads jar files located inside \$JAVA_HOME/jre/lib/ext.
- System/Application Classloader: the child of the Extension class loader; loads class files from the classpath, which defaults to the current directory.

Q3. What do you mean by inheritance in Java?

Inheritance is a mechanism through which an object acquires the properties and behavior of another class. It is used for method overriding and code reusability — new classes are built upon existing ones, so fields and methods of the parent class can be reused, and new members can be added.

Java supports five types of inheritance: Hybrid, Hierarchical, Single-level, Multi-level, and Multiple (the last one only through interfaces).

Q4. Why can we not override the static method in Java?

(a) A static method belongs to the class level, not the object level — method overriding is resolved based on the object.

(b) For a static method, the type reference decides which method to call without referring to the object, so the call is resolved at compile time.

If a child class defines a static method with the same signature as the parent class, the child's method hides the parent's method rather than overriding it.

Q5. What is Dynamic Method Dispatch in Java?

Dynamic Method Dispatch is the process through which a call to an overridden method is resolved at runtime rather than compile time. It is the actual object being referred to — not the type of the reference variable — that decides which version of the overridden method is executed.

Q6. What is a Java ClassPath?

A Java ClassPath is an environment variable that the JVM uses to locate all the classes required by a program.

Q7. What is the volatile keyword in Java, and what is its purpose?

A variable marked volatile is always read from and written to main memory instead of a thread's local cache, so updates made by one thread are immediately visible to other threads. It provides a lighter form of synchronization than the synchronized keyword, making it useful when you need visibility guarantees across threads without the overhead of locking.

Q8. Where does the finally block not execute in Java?

There is only one common case: the finally block does not execute if `System.exit(0)` is called inside the try or catch block.

Q9. What are the major points of distinction between StringBuffer and StringBuilder in Java?

StringBuffer is synchronized and thread-safe — two threads cannot call its methods at the same time — but this makes it comparatively slower.

StringBuilder is not synchronized and not thread-safe, which makes it faster and the better choice in a single-threaded environment. StringBuffer is preferred when shared data is modified across multiple threads.

Q10. What is the difference between Vector and ArrayList in Java?

- (a) Vector is synchronized and thread-safe, while ArrayList is not.
- (b) Because Vector is synchronized, it is slower; because ArrayList is not synchronized, it is comparatively faster.

Q11. Why is String immutable in Java?

- String pool: a String created with double quotes is stored in the string literal pool and can be referenced by several variables — if String were mutable, changing it through one reference would affect all of them.
- Classloading: String is used in the class-loading mechanism; if it were mutable, it could become a security risk.
- Cached hash value: since String is immutable, its hash code can be computed once and cached, which is efficient when String is used as a key in a HashMap or similar collection.

Q12. What is the difference between HashMap and HashSet in Java?

HashMap implements the Map interface, mapping keys to values. It is not synchronized/not thread-safe, allows one null key and multiple null values, and does not allow duplicate keys.

```
HashMap<Integer,String> studentMap = new HashMap<>();
studentMap.put(1, "Anushka");
studentMap.put(2, "Akansha");
```

HashSet implements the Set interface, which does not allow duplicate values. It is also not synchronized and not thread-safe.

```
HashSet<String> studentSet = new HashSet<>();
studentSet.add("Anushka");
studentSet.add("Akansha");
studentSet.add("Anjana");
```

Q13. What is the Producer-Consumer Problem, and what are the advantages of the Producer-Consumer Pattern?

The producer must wait if the shared bucket/queue is full, and the consumer must wait if it is empty. The classical solution uses wait() and notify() to coordinate the producer and consumer threads and block each of them on their own condition (queue full / queue empty).

Advantages: the producer does not need to know about the consumer(s) or how many there are (and vice versa); producer and consumer can work at different speeds, and more consumers can be added to match a faster producer; separating the two concerns leads to cleaner, more maintainable code.

Q14. How do you create an immutable class in Java?

- (a) Make the class final so it cannot be extended.
- (b) Make instance variables private (and final) so they cannot be accessed or reassigned from outside.
- (c) Do not provide setter methods.
- (d) Initialise variables in the constructor, and deep-copy any mutable objects passed in.
- (e) Return a clone of any mutable object from getter methods rather than the original reference, so the internal state stays protected.

Q15. What is the difference between the fail-fast and fail-safe iterator in Java?

A fail-fast iterator fails as soon as it detects that the collection's structure has changed during iteration (elements added, removed, or updated by another thread). This is implemented via a modification count; when the iterating thread notices the count has changed, it throws a ConcurrentModificationException.

A fail-safe iterator does not throw an exception when the collection is structurally modified during iteration — it works on a clone of the collection instead of the original.

Q16. What is an interface, and what are its key features?

An interface is a reference type, similar to a class, that can contain only constants, method signatures, default methods, static methods, and nested types — it cannot contain instance fields or constructors.

Key features: a class can implement multiple interfaces (multiple inheritance); methods are abstract by default unless marked default or static; interfaces can have default methods (Java 8+) and static methods (Java 8+).

Q17. What are the key principles of SOLID design?

- Single Responsibility Principle (SRP): a class should have only one reason to change.
- Open/Closed Principle (OCP): entities should be open for extension but closed for modification.
- Liskov Substitution Principle (LSP): objects of a superclass should be replaceable with objects of a subclass without breaking correctness.

- Interface Segregation Principle (ISP): clients should not be forced to depend on interfaces they do not use — prefer smaller, more specific interfaces.
- Dependency Inversion Principle (DIP): high-level modules should not depend on low-level modules; both should depend on abstractions.

Q18. What is a Java package?

A package is a namespace that organizes a set of related classes and interfaces, helping to avoid name conflicts and manage large projects. Packages can be built-in (java.util, java.lang, etc.) or user-defined, and are declared with the package keyword at the top of a source file.

Q19. What is the difference between a Set and a List in Java?

Set: does not allow duplicate elements and is unordered. Common implementations: HashSet, LinkedHashSet, TreeSet.

List: an ordered collection that allows duplicates and supports index-based access. Common implementations: ArrayList, LinkedList.

Q20. Explain the concept of Java Generics.

Generics let types (classes and interfaces) be used as parameters when defining classes, interfaces, and methods. This gives compile-time type safety and removes the need for casting — for example, List<String> guarantees only String objects can be added. Generics improve code reusability and maintainability.

Q21. What is the synchronized keyword, and what does synchronization achieve in Java?

The synchronized keyword controls access to a block of code, a method, or an object by multiple threads, ensuring only one thread can execute the synchronized block/method at a time — it can be applied to whole methods or specific blocks.

More broadly, synchronization is the mechanism that controls access to shared resources across threads to prevent data inconsistency. Besides the synchronized keyword, it can also be achieved with higher-level constructs such as Lock and ReadWriteLock. Proper synchronization is essential for data integrity in multithreaded programs.

Q22. What are lambda expressions in Java?

Introduced in Java 8, a lambda expression is a concise way to represent an instance of a functional interface (an interface with a single abstract method). The syntax consists of parameters, the arrow operator (->), and a body.

Example: (x, y) -> x + y — a function that takes two parameters and returns their sum.

Q23. What is a functional interface?

A functional interface is an interface that contains exactly one abstract method (it may still have multiple default or static methods). It can be used as the assignment target for a lambda expression or a method reference. Runnable is a common example.

Q24. Explain the difference between throw and throws.

`throw` is used to explicitly throw an exception from a method or block of code, e.g.
`throw new IllegalArgumentException("Invalid argument");`

`throws` is used in a method declaration to indicate that the method may throw one or more exceptions, informing callers that they need to handle them, e.g.
`public void myMethod() throws IOException, SQLException { ... }`

Q25. What is the Java Map interface?

The Map interface represents a collection of key-value pairs, where each key is unique and maps to exactly one value. Common implementations:

- HashMap: unordered, allows null keys/values, constant-time performance for basic operations.
- LinkedHashMap: like HashMap but maintains insertion order.
- TreeMap: implements SortedMap, storing keys in sorted order.

Q26. What is the difference between HashMap and Hashtable?

HashMap allows null keys and values, is not synchronized (not thread-safe), and generally offers better performance.

Hashtable does not allow null keys or values, is synchronized (thread-safe), and is considered legacy – it has largely been replaced by HashMap (or ConcurrentHashMap for thread safety).

Q27. What is the default keyword in interfaces?

The default keyword, introduced in Java 8, lets an interface method carry a default implementation. This allows new methods to be added to interfaces without breaking classes that already implement them — implementing classes can either override the default method or use it as-is.

Q28. What are the types of inner classes in Java?

- Non-static Inner Class: tied to an instance of the outer class; can access its members directly.
- Static Nested Class: behaves like a static member; does not need an outer class instance and can only access static members of the outer class.
- Method-local Inner Class: defined within a method; can access local variables/parameters of that method if they are final or effectively final.
- Anonymous Inner Class: a class without a name, defined at the point of instantiation; often used for a one-time implementation of an interface or subclass.

Q29. Explain the concept of method references in Java.

A method reference is shorthand for a lambda expression that just calls an existing method. There are four types:

- Static method reference: `ClassName::staticMethodName`
- Instance method reference of a particular object: `instance::instanceMethodName`
- Instance method reference of an arbitrary object of a particular type: `ClassName::instanceMethodName`
- Constructor reference: `ClassName::new`

Q30. What are the main features introduced in Java 8?

- Lambda Expressions — concise syntax for functional interfaces.
- Streams API — functional-style operations on sequences of elements.
- Default Methods in Interfaces — methods with a body inside an interface.
- Optional class — a way to avoid null references and represent optional values.
- New Date and Time API (java.time) — more comprehensive date/time handling.

Q31. Explain the use of the try-with-resources statement.

Introduced in Java 7, try-with-resources manages resources that must be closed after use, such as files or sockets. Any resource implementing `AutoCloseable` can be declared inside the parentheses of the try statement, and it is closed automatically at the end of the block, preventing resource leaks.

```
try (BufferedReader br = new BufferedReader(new FileReader("file.txt"))) {  
    // use br  
} catch (IOException e) {  
    // handle exception  
}
```

Q32. What is the difference between Comparable and Comparator?

`Comparable` defines the natural ordering of objects — you implement `compareTo()` inside the class itself, and it is used when sorting objects of that one class.

`Comparator` defines an external ordering — you implement `compare()` in a separate class or lambda, which lets you sort objects of different classes, or the same class in different ways.

Q33. What is the significance of the instanceof operator?

`instanceof` tests whether an object is an instance of a specific class or subclass, returning true or false. It's particularly useful for checking an object's type before casting, helping to avoid a `ClassCastException`.

Q34. Explain the concept of the super keyword.

`super` refers to the immediate parent class of the current object. It is used to access the parent class's methods and fields, and to invoke the parent class's constructor — `super()` can be called within a subclass constructor to invoke the parent constructor.

Q35. What is the finalize() method?

`finalize()` is a method on the `Object` class that the garbage collector calls just before reclaiming an object, allowing it to perform cleanup such as releasing resources. Its use is discouraged because there's no guarantee when — or even if — it will be called, which makes resource management unpredictable.

Q36. What are the main interfaces of the Java Collections Framework?

- `Collection`: the root interface for all collections.
- `List`: an ordered collection that allows duplicates.

- Set: a collection that does not allow duplicates.
- Map: a collection of key-value pairs.
- Queue: a collection designed for holding elements prior to processing.

Q37. What is a Thread in Java?

A Thread is a lightweight subprocess — the smallest unit of processing — that allows concurrent execution of parts of a program. Java provides the Thread class and the Runnable interface for creating and managing threads; multithreading improves an application's performance and responsiveness.

Q38. How do you create a thread in Java?

There are two main ways:

1) Extend the Thread class:

```
class MyThread extends Thread { public void run() { ... } }  
MyThread thread = new MyThread();  
thread.start();
```

2) Implement the Runnable interface:

```
class MyRunnable implements Runnable { public void run() { ... } }  
Thread thread = new Thread(new MyRunnable());  
thread.start();
```

Q39. What is a deadlock?

A deadlock occurs when two or more threads are blocked forever, each waiting on a resource held by the other. For example, Thread A holds Resource 1 and waits for Resource 2, while Thread B holds Resource 2 and waits for Resource 1 — neither can proceed. Deadlocks can be avoided with techniques such as lock ordering, timeouts, or higher-level concurrency utilities like those in `java.util.concurrent`.

Q40. What is the purpose of the clone() method?

`clone()` creates a copy of an object. The `Object` class provides `clone()`, but a class must implement the `Cloneable` marker interface and override `clone()` to support duplication. By default, `clone()` performs a shallow copy, though it can be overridden to perform a deep copy when needed.

Q41. What is a Queue in Java?

A Queue is a collection designed for holding elements prior to processing, following FIFO (First-In-First-Out) — elements are added at the end and removed from the front. Implementations include `LinkedList`, `PriorityQueue`, and `ArrayDeque`, each with different behaviors and performance characteristics.

Q42. What is the difference between shallow copy and deep copy?

Shallow copy: a new object is created, but the objects it contains are not copied — only their references are copied, so changes to the original nested objects also affect the copy.

Deep copy: a new object is created and all contained objects are recursively copied too, producing a fully independent copy — changes to the original do not affect the deep copy.

Q43. What are Java annotations used for?

Annotations are metadata about code that don't affect its execution. They're used for:

- Compile-time checks — e.g., `@Override` helps the compiler catch mistakes.
- Runtime processing — frameworks like Spring or Hibernate read annotations to alter behavior.
- Documentation — e.g., generating docs with Javadoc.

Q44. What is the Java Memory Model?

The Java Memory Model (JMM) defines how threads interact through memory and how one thread's changes become visible to others. It specifies rules for visibility, atomicity, and ordering of operations, which is essential for writing correct and efficient concurrent programs.

Q45. Explain the concept of reflection in Java.

Reflection lets a program inspect and manipulate classes, methods, fields, and other components at runtime, using the `java.lang.reflect` package. It's useful for tasks like object serialization, debugging, and building frameworks, but should be used judiciously because of performance overhead and security concerns.

Q46. What is the purpose of the `join()` method in threading?

```
join() pauses execution of the current thread until the thread on which it was called has completed – useful when one thread needs to wait for another to finish before continuing.  
Thread thread = new Thread(new MyRunnable());  
thread.start();  
thread.join(); // waits for the thread to finish
```

Q47. What are the benefits of using the Java Stream API?

- Conciseness: more readable code written in a functional style.
- Lazy evaluation: elements are processed only when necessary, which can improve performance.
- Parallel processing: streams can be processed in parallel with minimal effort, improving performance on multicore processors.
- Declarative approach: focuses on what to do rather than how to do it.

Q48. What is the difference between `==` and `equals()` when comparing two objects?

`==` checks whether two references point to the same object in memory — it compares memory addresses. `equals()` checks whether two objects are logically equivalent, and can be overridden in a custom class to define what equality means for that class.